C++ Notes: Return Statement

(Returning values from functions)

Author: Ba Tung Le

Learning Objectives

After this section, you will be able to:

- Explain what it means for a function to **return** a value
- Choose the correct **return type** (int, double, string, ...)
- Use `return` to send a result back to the caller
- Store the returned value in a variable or print it directly
- Understand that `return` immediately exits the function

Core Idea

Core idea

A function can **receive** information (parameters), and it can also **give back** information (a return value).

1. Return Type vs. `void`

Return type

The return type tells C++ what kind of value the function will return.

- `void` means the function returns nothing
- `int`, `double`, `string`, `char`, ... mean the function returns a value of that type

2. Example: Cube a Number

Goal

Create a function `cube(num)` that returns `num * num * num`.

Version A: Using an intermediate variable

Listing 1: cube() returns a double

```
double cube(double num) {
  double result = num * num * num;
  return result;
```

```
}
```

Calling the function and storing the result

Listing 2: Store returned value in a variable

```
int main() {
  double answer = cube(5.0);
  cout << answer << endl;
  return 0;
}
```

Output:

```
125
```

3. Print the Return Value Directly

You do not always need a separate variable.

Listing 3: Print returned value directly

```
int main() {
  cout << cube(5.0) << endl;
  return 0;
}
```

Output:

```
125
```

4. Version B: Return the expression directly

Listing 4: Return the expression directly

```
double cube(double num) {
  return num * num * num;
}
```

This produces the same result, just shorter.

5. Important: `return` immediately exits the function

Key behavior

When C++ reaches `return`, it stops executing the function immediately. Any code after `return` will never run.

Example: Code after return is unreachable

Listing 5: Unreachable code after return

```
double cube(double num) {
  return num * num * num;
  cout << "Hello" << endl; // never runs
}
```

What happens?

- The function returns the value right away.
- The `cout` line is never reached, so `"Hello"` is not printed.

Common Mistakes

Common mistakes

- Declaring a non-`void` function but forgetting to `return` a value
- Returning the wrong type (e.g., returning a string in an `int` function)
- Writing code after `return` and expecting it to run
- Confusing `return` with printing (return gives back a value; `cout` prints it)

Ba Tung Le

Email: tombn2607@gmail.com